GHRISTU, PUNE
School of Engineering and Technology

MINI PROJECT REPORT

on

Analysis of Factors Affecting Student Exam Scores

Data cleaning, wrangling, exploration and a simple prediction model

Submitted for the course
Data Science

Submitted by: Devansh Singh
Roll No. TB1-CE-07

Under the guidance of: Prof. Veena Kiragi

Academic year 2026-27

Declaration

I, Devansh Singh, declare that this mini project report titled “Analysis of Factors Affecting Student Exam Scores” is a record of work done by me as part of the Data Science course. The Python code, the figures, the tables and the dashboard in this report were prepared by me for this assignment. The dataset is a simulated practice dataset, and this is stated clearly in Chapter 2. Books, documentation and other sources that I used are listed in the references.

Devansh Singh
Roll No. TB1-CE-07

Acknowledgement

I would like to thank Prof. Veena Kiragi for explaining the steps of a data science project in class, and for suggesting that I should keep the cleaning work visible instead of hiding it behind the final model. I also thank my classmates for the discussions on how to handle missing values, and my family for their support during the work.

Abstract

Most data science work is spent on getting the data into a usable shape, and the modelling is a small part at the end. This mini project follows that full path on a small education dataset. The raw file has 534 rows and 12 columns and it contains problems that are common in hand entered survey data: the same category written in many ways, units typed inside number cells, repeated rows, empty answers and impossible values. These problems were found, counted and fixed using Python and pandas, which left 512 clean records.

After cleaning, new columns were created (study band, attendance band, score gain and grade) and the data was grouped and summarised to compare students. Exploratory analysis showed that the previous exam score has the strongest link with the final score (correlation 0.61), followed by weekly study hours (0.40). Sleep has a curved relationship with the score, so a new feature measuring the distance from 7.5 hours of sleep was added. A linear regression model trained on 80 percent of the data explained 62 percent of the variance in the final score on the unseen 20 percent, with a mean absolute error of 4.7 marks. A random forest was tried for comparison and did not do better. A simple web dashboard was built to show the results and to let a user try the model with their own inputs.

Contents

Declaration i

Acknowledgement ii

Abstract iii

1 Introduction 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 1

1.4 Scope 2

2 Tools and Dataset 3

2.1 Tools Used 3

2.2 About the Dataset 3

3 Methodology 5

4 Data Inspection and Cleaning 6

4.1 First Look at the Raw File 6

4.2 Problems Found and Actions Taken 6

4.2.1 Inconsistent labels 7

4.2.2 Units inside cells 7

4.2.3 Duplicates 8

4.2.4 Target column 8

4.2.5 Range checks 8

4.2.6 Missing values 8

4.2.7 Outlier check 8

4.3 Result of Cleaning 8

5 Data Wrangling **10**

5.1 Grouping 10

5.2 Pivot Table 11

6 Exploratory Data Analysis **12**

6.1 Distributions 12

6.2 Correlation 12

6.3 Study Hours and Score 13

6.4 Group Comparison 14

6.5 Sleep 14

7 Model Building **15**

7.1 Setup 15

7.2 Models 15

7.3 Measures Used 15

8 Results and Discussion **16**

8.1 Model Comparison 16

8.2 What the Model Says 17

8.3 Limitations 18

9 The Dashboard **19**

10 Conclusion and Future Scope **20**

10.1 Conclusion 20

10.2 Future Scope 20

References **21**

A Project Files and How to Run **22**

B Main Cleaning Code **23**

List of Figures

4.1 Number of empty cells per column in the raw file 6

6.1 Histograms of study hours, attendance, sleep hours and final score 12

6.2 Correlation matrix of the numeric columns 13

6.3 Study hours per week against the final score, with a straight line fitted . . . 13

6.4 Mean final score by study hours, attendance and parent education 14

8.1 Actual and predicted scores for the test students. Points on the line would be perfect predictions. 16

8.2 Effect of a one standard deviation increase in each input on the predicted score 17

List of Tables

2.1 Software used in the project 3

2.2 Columns in the raw dataset 4

4.1 Cleaning log 7

4.2 Summary statistics of the cleaned numeric columns 9

5.1 Columns created during wrangling 10

5.2 Mean final score by group 11

5.3 Mean final score by study band and private tuition 11

8.1 Performance on the 103 test students 16

8.2 Linear regression weights (intercept 14.74) 17

A.1 Files in the submission folder 22

1. Introduction

1.1 Background

Schools and colleges collect a lot of information about students, such as attendance, previous marks and study habits. Teachers often have an idea of what helps students do well, but it is useful to check these ideas against data. If some factors turn out to matter more than others, then advice to students can be more specific.

Data collected in practice is rarely tidy. Different people fill the same form in different ways, some fields are left empty and a few values are typed wrongly. Before any analysis, this has to be handled. For that reason the cleaning and wrangling steps get the most space in this report.

1.2 Problem Statement

Given a raw table of student records, prepare it for analysis and find out which factors are related to the final exam score. Then build a simple model that estimates the final score from those factors.

1.3 Objectives

1. To inspect a raw dataset and list the quality problems in it.
2. To clean the data using pandas: fix labels, convert text to numbers, remove duplicates, check ranges and fill missing values.
3. To wrangle the clean data by creating new columns, grouping, and building a pivot table.
4. To explore the data with summary statistics and plots.
5. To train and compare a linear regression model and a random forest model and judge them on data they have not seen.
6. To present the results in a short dashboard that a non technical person can read.

1.4 Scope

The project works on one table of about five hundred students. It does not use deep learning, text data or any external database. The aim is to show the data science workflow clearly and correctly, and not to build the most accurate model possible.

2. Tools and Dataset

2.1 Tools Used

Table 2.1: Software used in the project

Tool	Purpose
Python 3.12	Main programming language
pandas	Loading, cleaning and wrangling the table
NumPy	Numeric work and random number generation
matplotlib	Plots used in this report
scikit-learn	Train/test split, models and error measures
HTML, CSS and JavaScript	The dashboard
LaTeX	This report

2.2 About the Dataset

The file `raw_students.csv` holds one row for each student. The final score is out of 100. **The values in this file are simulated.** They were produced by the script `make_raw_data.py` with a fixed random seed, so that the same file can be created again. The script first creates sensible values and relationships between the columns, and then adds typing mistakes, missing answers and repeated rows on purpose, in the way that a real survey file would have them. Because of this, the conclusions of the project describe this practice dataset and they should not be taken as facts about real students.

The code does not depend on how the file was made. If a real file with the same column names is placed at `data/raw_students.csv`, the whole pipeline runs again without any change.

Table 2.2: Columns in the raw dataset

Column	Type	Meaning
student_id	text	Identifier such as S0328
gender	text	Male or female, written in many ways
age	number	Age in years (some cells contain “yrs”)
area	text	Urban or rural
parent_education	text	Highest education in the family: school, graduate or postgraduate
study_hours_week	number	Hours of self study per week
attendance	number	Class attendance in percent
sleep_hours	number	Average hours of sleep per night
internet_access	text	Whether the student has internet at home
private_tuition	text	Whether the student takes private tuition
previous_score	number	Marks in the previous exam, out of 100
final_score	number	Marks in the final exam, out of 100 (target)

3. Methodology

The work was done in the order shown below. Each step is a block of code in `analysis.py`, and every cleaning step writes a line to a log, which is used later in this report and in the dashboard.

1. **Load and inspect.** Read the CSV, check the shape, data types and the number of empty cells in each column.
2. **Clean.** Fix text labels, convert numbers stored as text, remove duplicate rows, handle the target column, check value ranges, and fill the missing values.
3. **Wrangle.** Create new columns, group the data and build a pivot table.
4. **Explore.** Summary statistics, histograms, correlations and group comparisons.
5. **Model.** Split the data, train a baseline, a linear regression and a random forest, and compare them using MAE, RMSE and R^2.
6. **Report.** Save the clean file, the figures and a results file, which is read by the dashboard.

4. Data Inspection and Cleaning

4.1 First Look at the Raw File

The raw file has 534 rows and 12 columns. On loading, three columns that should hold numbers (`age`, `study_hours_week` and `attendance`) were read as text, which was the first sign of trouble. Figure 4.1 shows the empty cells found in each column. Attendance has the most (46), partly because the entries “N/A” are read as missing by pandas.

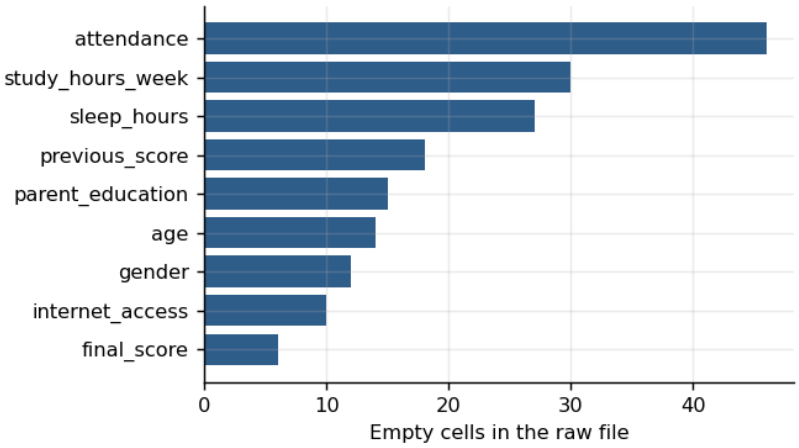


Figure 4.1: Number of empty cells per column in the raw file

4.2 Problems Found and Actions Taken

Table 4.1 lists every problem, how many cases there were, and what was done. The counts are printed by the program, they were not typed in by hand.

Table 4.1: Cleaning log

Problem	What was found	Action
Inconsistent labels	Gender had 10 spellings, parent education 11, internet access 8, tuition 8, area 4	Trimmed spaces, made lower case and mapped to 2, 3, 2, 2 and 2 labels
Units inside cells	194 cells such as 78%, 14.3 hrs, 16 yrs	Removed the unit text and converted to numbers
Duplicate rows	14 rows repeated completely	Kept the first copy
Bad target values	8 rows with final score missing or outside 0 to 100	Dropped, because the target should not be guessed
Impossible values	Study hours 4, attendance 3, sleep 3, previous score 2 outside valid limits	Set to missing, then filled
Missing values	173 empty cells in 8 columns	Median for numbers, most common label for text

4.2.1 Inconsistent labels

The same answer was written in different ways. For example, gender appeared as `Male`, `male`, `M`, `MALE` (with a space) and `m`. A group count on such a column would split one group into five. The text was stripped and made lower case, and then a dictionary mapped each variant to one label.

```
gmap = {"male": "Male", "m": "Male", "female": "Female", "f": "Female"}
df["gender"] = df["gender"].str.strip().str.lower().map(gmap)
```

A useful side effect of using a dictionary is that any variant that was not expected becomes missing, so it shows up in the missing value count instead of staying hidden.

4.2.2 Units inside cells

Some attendance values were written as `78%`, some study hours as `14.3 hrs` and some ages as `16 yrs`. A regular expression removed everything except digits, the decimal point and the minus sign, and `pd.to_numeric` converted the result.

```
txt = df[c].astype("string").str.replace(r"[^0-9.\-]", "", regex=True)
df[c] = pd.to_numeric(txt.replace("", pd.NA), errors="coerce").astype(float)
```

4.2.3 Duplicates

There were 14 rows that were exact copies of other rows. Student IDs were checked after removing them, and each ID now occurs only once.

4.2.4 Target column

The final score is the value to be predicted, so rows with a missing final score were removed instead of being filled. Two rows had scores of 250 and -10, which are not possible for a test marked out of 100, and these were removed as well. In total 8 rows were dropped, which leaves 512 rows.

4.2.5 Range checks

For each numeric column a valid range was decided from common sense, and values outside it were set to missing. The ranges were 0 to 60 hours for study, 0 to 100 for attendance and previous score, and 3 to 12 hours for sleep. This caught values such as 168 study hours in a week, an attendance of 940 and a sleep time of 24 hours.

4.2.6 Missing values

After the earlier steps, 173 cells were still empty. Numeric columns were filled with the median, because the median is not pulled by extreme values. Text columns were filled with the most common label. The medians and modes were calculated once and stored, so they can be reported. Filling values keeps the number of rows, but it also makes the data a little less varied than it really is, which is mentioned again in the limitations.

4.2.7 Outlier check

After the range checks, the IQR rule (values beyond 1.5 times the interquartile range from the quartiles) was applied to see what is still unusual. It flagged 9 values in study hours, 3 in attendance, 8 in sleep, 3 in previous score and 2 in final score. These values are possible in real life (for example a student who studies 24 hours a week), so they were kept.

4.3 Result of Cleaning

The clean table has 512 rows. Table 4.2 gives the summary statistics of the numeric columns.

Table 4.2: Summary statistics of the cleaned numeric columns

Column	Mean	Std	Min	25%	50%	75%	Max
age	16.46	0.95	15.0	16.0	16.0	17.0	18.0
study_hours_week	11.15	4.19	0.5	8.6	11.1	13.7	24.9
attendance	81.66	9.89	53.0	75.0	82.0	88.0	100.0
sleep_hours	6.95	0.98	4.0	6.3	7.0	7.6	9.9
previous_score	63.40	13.33	23.0	54.0	63.0	72.0	98.0
final_score	71.41	9.96	42.0	65.0	72.0	79.0	99.0

5. Data Wrangling

Wrangling means reshaping the clean data so that questions can be asked of it. Four new columns were added.

Table 5.1: Columns created during wrangling

New column	How it is made
study_band	Low (up to 7 hours), medium (7 to 14 hours), high (over 14 hours)
attendance_band	Below 70%, 70 to 85%, above 85%
gain	Final score minus previous score
grade	Fail (below 40), pass (40 to 59), merit (60 to 74), distinction (75 and above)

The bands were made with `pd.cut`. The number of students who fall in each grade is 64 for pass, 246 for merit and 202 for distinction. Nobody falls in the fail grade, because the lowest final score in this data is 42. The average gain is 8.0 marks, so on the whole the students improved from the previous exam.

5.1 Grouping

The mean final score was found for each group with `groupby`. Table 5.2 shows the results.

Table 5.2: Mean final score by group

Grouped by	Group	Students	Mean score
Study hours	Low	85	65.5
	Medium	309	71.0
	High	118	76.7
Attendance	Below 70%	65	67.4
	70 to 85%	274	70.9
	Above 85%	173	73.7
Parent education	School	190	70.4
	Graduate	219	71.8
	Postgraduate	103	72.3
Private tuition	No	309	70.5
	Yes	203	72.8
Internet access	No	114	69.6
	Yes	398	71.9
Gender	Female	239	72.0
	Male	273	70.9
Area	Rural	205	71.3
	Urban	307	71.5

5.2 Pivot Table

A pivot table with study band in the rows and tuition in the columns shows the effect of two factors together (Table 5.3).

Table 5.3: Mean final score by study band and private tuition

Study band	No tuition	Tuition
Low	64.5	67.0
Medium	69.8	73.0
High	76.7	76.7

Tuition seems to help students who study few or medium hours. For students who already study more than 14 hours a week, the two averages are the same.

6. Exploratory Data Analysis

6.1 Distributions

Figure 6.1 shows the histograms of four columns. All four are roughly bell shaped. The final score is centred near 72. Attendance is cut off at 100 percent, so there is a small pile of students at the top.

One thing to notice is the single tall bar at the middle of study hours, attendance and sleep (near 11 hours, 82 percent and 7 hours). This is a side effect of filling missing values with the median: every filled cell gets exactly the same number, so that bar is higher than its neighbours. It is a small price for keeping the rows, but it is a good example of how a cleaning choice leaves a mark on the data.

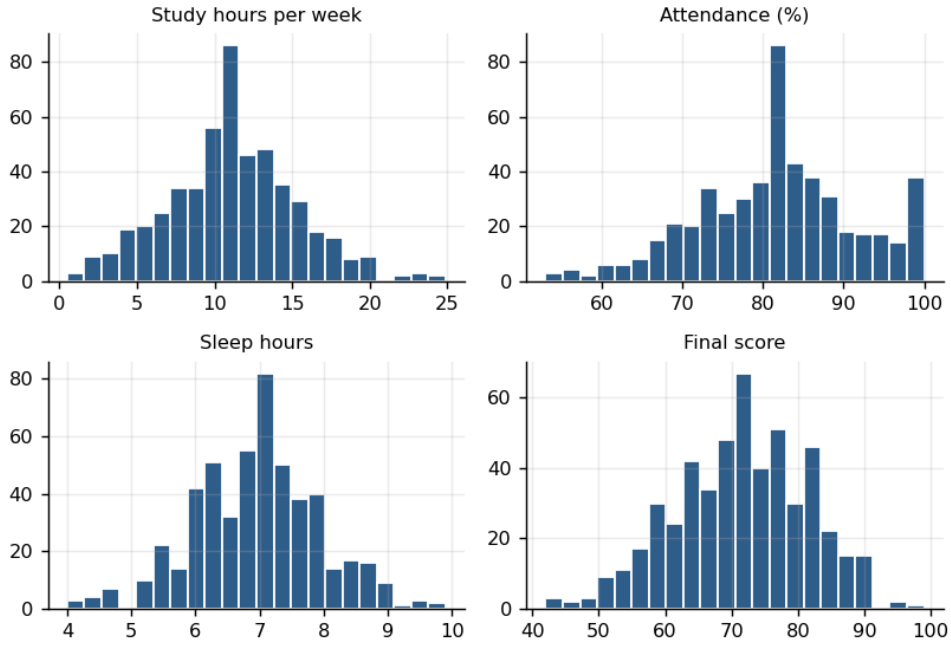


Figure 6.1: Histograms of study hours, attendance, sleep hours and final score

6.2 Correlation

Figure 6.2 is the correlation matrix of the numeric columns. With the final score, the previous score has a correlation of 0.61, study hours 0.40, attendance 0.21 and sleep 0.14. Age has no correlation (0.00), and for that reason it was left out of the model. Gender and area also differ by only about one mark or less between groups (Table 5.2), so they were left out

as well.

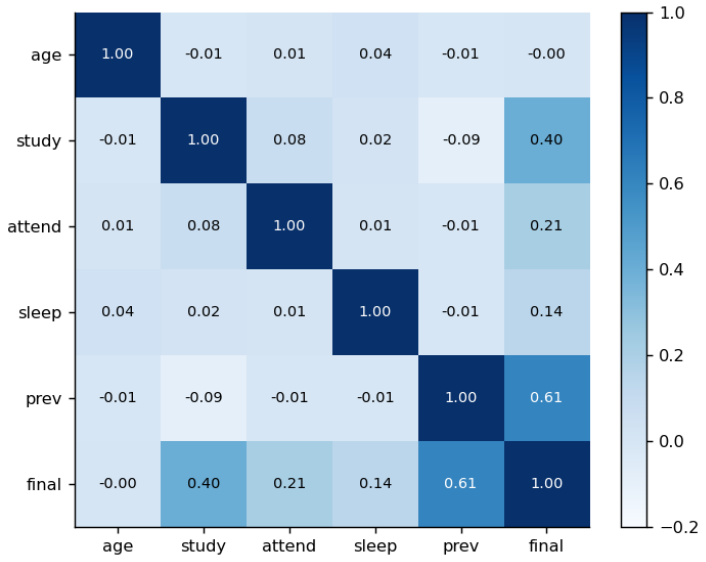


Figure 6.2: Correlation matrix of the numeric columns

6.3 Study Hours and Score

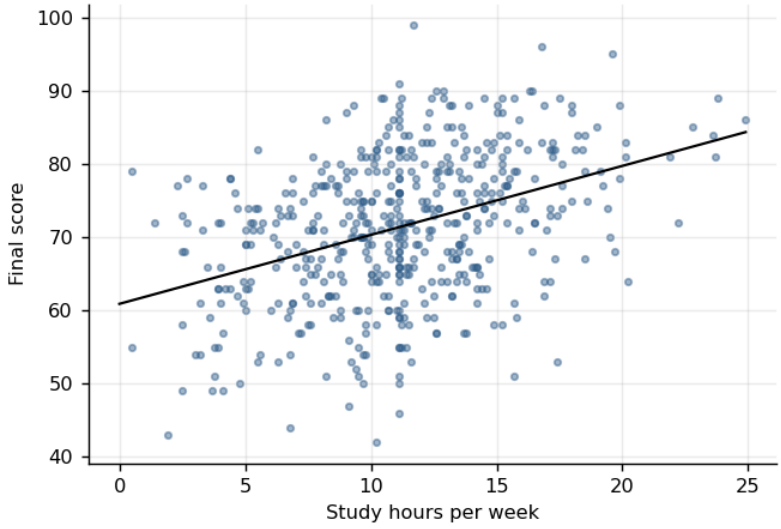


Figure 6.3: Study hours per week against the final score, with a straight line fitted

The points in Figure 6.3 rise from left to right, but they are spread widely. This means study hours matter, and that other factors, like the previous score, also play a part.

6.4 Group Comparison

Figure 6.4 shows the group means of Table 5.2 for three factors. The bars start from zero on purpose, so that the size of the differences is not exaggerated. Study hours give the largest difference between groups (about 11 marks from low to high). Parent education gives only about 2 marks.

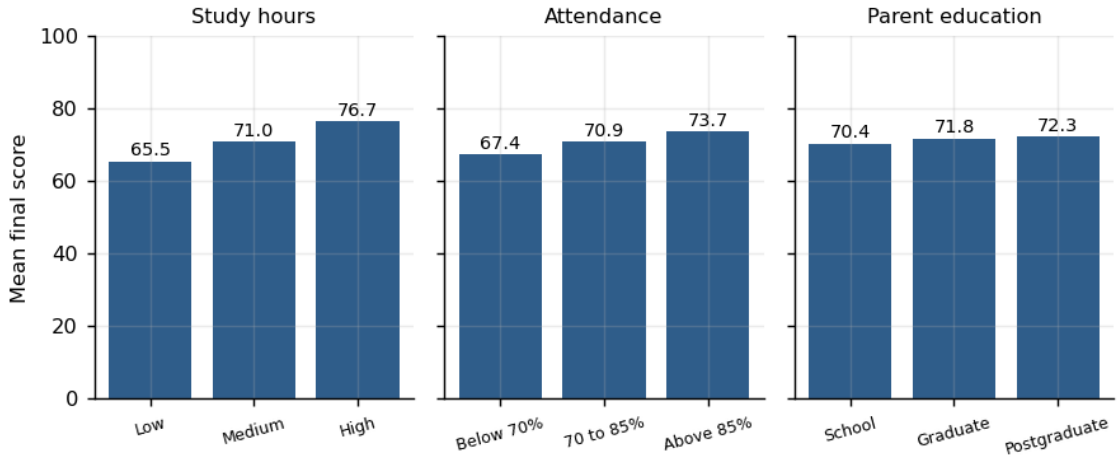


Figure 6.4: Mean final score by study hours, attendance and parent education

6.5 Sleep

Sleep did not behave like the other factors. When students were grouped by sleep time, the mean score was 65.6 for 5.5 hours or less, 70.0 for 5.5 to 6.5 hours, 72.9 for 6.5 to 7.5 hours, 72.8 for 7.5 to 8.5 hours, and 70.1 for more than 8.5 hours. The score goes up and then comes down again, so both short and long sleep are linked with lower marks. A straight line cannot describe this pattern. A new feature called `sleep_gap`, the absolute difference between sleep hours and 7.5, was created to let a linear model use it. The value 7.5 was chosen by looking at the group means above.

7. Model Building

7.1 Setup

The goal is to estimate the final score (a number), so this is a regression problem. The inputs were study hours, attendance, sleep gap, previous score, tuition (0 or 1), internet access (0 or 1), and two indicator columns for parent education (graduate and postgraduate, with school as the reference group).

The data was split at random into 409 rows for training (80 percent) and 103 rows for testing (20 percent), with a fixed seed of 42. The test rows were not used while fitting the models.

7.2 Models

- **Baseline.** Always predicts the mean of the training scores. Any real model should beat this.
- **Linear regression.** Fits one weight for each input. It is simple, and the weights can be explained.
- **Random forest.** An average of 300 decision trees (minimum 5 samples in a leaf). It can follow curved patterns without being told.

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42)
lr = LinearRegression().fit(Xtr, ytr)
rf = RandomForestRegressor(300, min_samples_leaf=5, random_state=42).fit(Xtr, ytr
    )
```

7.3 Measures Used

MAE is the average size of the error in marks. **RMSE** also measures the error in marks, but it punishes large mistakes more. R^2 is the share of the variation in the final score that the model explains, where 0 means no better than the mean and 1 means perfect.

8. Results and Discussion

8.1 Model Comparison

Table 8.1: Performance on the 103 test students

Model	MAE	RMSE	R^2
Mean baseline	8.13	10.05	0.000
Linear regression	4.74	6.16	0.622
Random forest	5.03	6.40	0.592

Both models are much better than the baseline. The simple linear regression is slightly better than the random forest on every measure, so it was chosen. Its typical error is about 4.7 marks. To check that the result does not depend on one lucky split, a 5 fold cross validation was run on the full data. The R^2 values of the folds were 0.661, 0.612, 0.608, 0.638 and 0.729, with a mean of 0.650. This is close to the test score, which is a good sign.

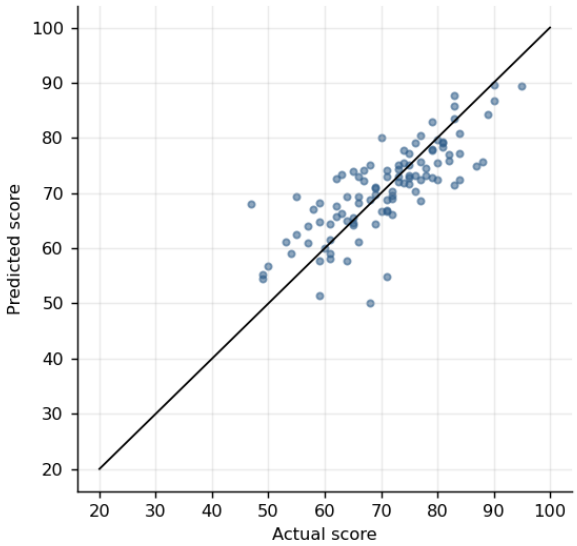


Figure 8.1: Actual and predicted scores for the test students. Points on the line would be perfect predictions.

8.2 What the Model Says

Table 8.2 gives the weights of the linear regression. Each weight is the change in the predicted score when that input goes up by one unit and the others stay the same. Because the inputs have different units, the last column rescales each effect to a one standard deviation change, so they can be compared.

Table 8.2: Linear regression weights (intercept 14.74)

Input	Weight per unit	Effect of 1 std. dev.
previous_score	0.48	6.43
study_hours_week	1.02	4.27
sleep_gap	−3.00	−2.05
attendance	0.18	1.74
private_tuition	2.26	1.11
internet_access	2.08	0.86
parent_postgraduate	1.21	0.48
parent_graduate	0.61	0.30

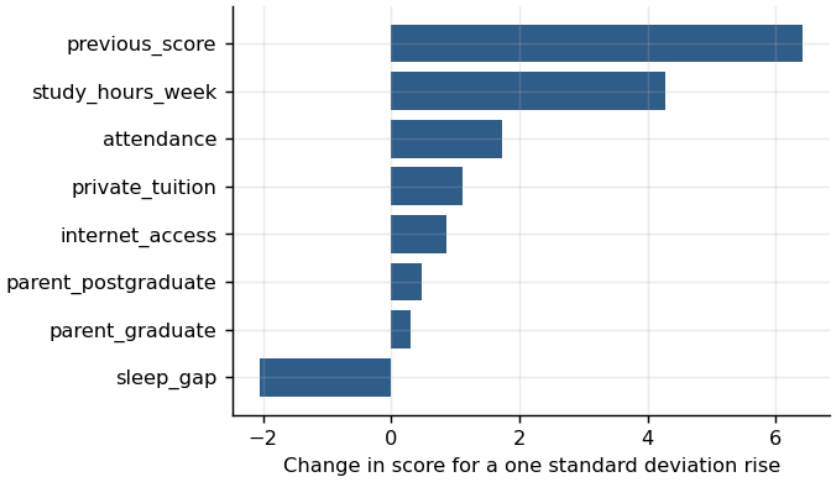


Figure 8.2: Effect of a one standard deviation increase in each input on the predicted score

The main points are:

- The previous score is the strongest input. A student who scored 10 marks more in the last exam is expected to score about 4.8 marks more in this one.
- Each extra hour of study per week is linked with about one extra mark.
- Each hour of sleep away from 7.5 hours costs about 3 marks.

- Attendance, tuition and internet access have smaller but positive weights.
- Parent education has a small effect once the other inputs are in the model.

The random forest ranks the inputs in the same order at the top: previous score (0.54), study hours (0.28), sleep gap (0.08) and attendance (0.08). The other four inputs share less than 0.04 of the importance.

8.3 Limitations

- The dataset is simulated, and it has only 512 rows. A real school dataset would be messier and could give different results.
- The model finds links, not causes. It does not prove that more study hours will raise a particular student's marks.
- Filling missing values with the median makes the columns slightly less spread out than the real values would be.
- The ranges used in the range checks and the 7.5 hour sleep point were chosen by hand, and different choices would change the results a little.
- No tuning of the models was done, and only one random split was used for the main comparison.

9. The Dashboard

A single page web dashboard was built to show the project to a person who has not read the code. It reads `results.json`, which is written by `analysis.py`, so the page shows the numbers produced by the analysis, not numbers typed into the page. It has four parts.

1. A summary of the cleaning: rows before and after, and the log of each problem and fix.
2. A scatter plot of study hours against final score with the fitted line, and a chart of average scores for each group.
3. The comparison of the three models.
4. A form where the user enters study hours, attendance, sleep, previous score, tuition, internet access and parent education. The page applies the linear regression weights from Table 8.2 and shows the estimated score. The inputs are limited to the ranges that were present in the data, because the model should not be trusted outside them.

10. Conclusion and Future Scope

10.1 Conclusion

The project went through the main steps of a data science task on one dataset. A raw file with many typical problems was inspected and cleaned, and every fix was counted. The clean data was wrangled with new columns, grouping and a pivot table. The analysis showed that the previous score and weekly study hours are the strongest factors for the final score, that attendance, tuition and internet access add a little, and that sleep helps up to about 7.5 hours and then stops helping. A linear regression explained about 62 percent of the variation in the final score on unseen data, and it was more useful than the random forest here because it was equally accurate and easy to explain.

The most important thing learned was that the cleaning stage changed the data a lot (22 rows removed, 194 cells reformatted, 173 cells filled), and a model trained on the raw file would have been unreliable.

10.2 Future Scope

- Run the same pipeline on a real, public student performance dataset.
- Compare other imputation methods, such as filling by group, and see whether the results change.
- Add subject wise marks and predict whether a student will pass, as a classification task.
- Tune the models with grid search and use cross validation for the comparison.

Bibliography

[1] W. McKinney, “Data Structures for Statistical Computing in Python,” *Proceedings of the 9th Python in Science Conference*, 2010.

[2] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825 to 2830, 2011.

[3] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90 to 95, 2007.

[4] G. James, D. Witten, T. Hastie and R. Tibshirani, *An Introduction to Statistical Learning*, Springer.

[5] A. Geron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, O’Reilly Media.

A. Project Files and How to Run

Table A.1: Files in the submission folder

File	Contents
src/make_raw_data.py	Creates the simulated raw dataset
src/analysis.py	Cleaning, wrangling, exploration and models
data/raw_students.csv	Raw data, 534 rows
data/clean_students.csv	Clean data, 512 rows and 16 columns
data/results.json	Numbers used by the dashboard
figures/	Plots used in this report
dashboard.html	The dashboard

To run the project, install Python with pandas, NumPy, matplotlib and scikit-learn, then run these commands from the project folder:

```
python src/make_raw_data.py
python src/analysis.py
```

B. Main Cleaning Code

```
# duplicates
dups = int(df.duplicated().sum())
df = df.drop_duplicates().reset_index(drop=True)

# target column
bad_target = df["final_score"].isna() | ~df["final_score"].between(0, 100)
df = df[~bad_target].reset_index(drop=True)

# impossible values become missing
rules = {"study_hours_week": (0, 60), "attendance": (0, 100),
         "sleep_hours": (3, 12), "previous_score": (0, 100), "age": (10, 25)}
for c, (lo, hi) in rules.items():
    bad = df[c].notna() & ~df[c].between(lo, hi)
    df.loc[bad, c] = np.nan

# fill missing values
for c in num_cols:
    df[c] = df[c].fillna(df[c].median())
for c in cat_cols:
    df[c] = df[c].fillna(df[c].mode()[0])

# new columns
df["study_band"] = pd.cut(df["study_hours_week"], [-0.1, 7, 14, 100],
                          labels=["Low", "Medium", "High"])
df["gain"] = df["final_score"] - df["previous_score"]
```